Prototype Pattern

The **Prototype Pattern** is a creational design pattern used to clone existing objects instead of constructing them from scratch. It enables efficient object creation, especially when the initialization process is complex or costly.

Formal Definition:
"Prototype pattern creates duplicate objects while keeping performance in mind. It provides a mechanism to copy the original object to a new one without making the code dependent on their classes."

In simpler terms:
Imagine you already have a perfectly set-up object — like a well-written email template or a configured game character. Instead of building a new one every time (which can be repetitive and expensive), you just **copy the existing one** and make small adjustments. This is what the Prototype Pattern does. It allows you to create new objects by copying existing ones, saving time and resources.

Real-life Analogy (Photocopy Machine)

Think of preparing ten offer letters. Instead of typing the same letter ten times, you write it once, photocopy it, and change just the name on each copy. This is how the Prototype Pattern works: start with a base object and produce modified copies with minimal changes.

Understanding

Let's understand better through a common challenge in software systems.

Consider an email notification system where each email instance

requires extensive setup—loading templates, configurations, user settings, and formatting. Creating every email from scratch introduces redundancy and inefficiency.

Now imagine having a **pre-configured prototype email**, and simply cloning it for each user while modifying a few fields (like the name or content). That would save time, reduce errors, and simplify the logic.

Suitable Use Cases

Apply the Prototype Pattern in these situations:

- Object creation is resource-intensive or complex.
- You require many similar objects with slight variations.
- You want to avoid writing repetitive initialization logic.
- You need runtime object creation without tight class coupling.

Real-life Example

Imagine we're building a Email Template System at TUF:

Bad Code: Incomplete Use of Design Principles

```
#include <bits/stdc++.h>

using namespace std;


// Target Interface

// Abstract class to represent an Email Template

class EmailTemplate {
```

```
public:
    virtual void setContent(const string& content) = 0;
    virtual void send(const string& to) = 0;
    virtual ~EmailTemplate() {}
};

// A concrete email class, hardcoded
class WelcomeEmail : public EmailTemplate {
private:
    string subject;
    string content;

public:
    WelcomeEmail() {
        subject = "Welcome to TUF+";
        content = "Hi there! Thanks for joining us.";
    }

    void setContent(const string& content) override {
        this->content = content;
    }
```

```
    void send(const string& to) override {
        cout << "Sending to " << to << ": [" << subject << "] " << content << endl;
    }
};

int main() {
    // Create a welcome email
    WelcomeEmail email1;
    email1.send("user1@example.com");

    // Suppose we want a similar email with slightly different content
    WelcomeEmail email2;
    email2.setContent("Hi there! Welcome to TUF Premium.");
    email2.send("user2@example.com");

    // Yet another variation
    WelcomeEmail email3;
    email3.setContent("Thanks for signing up. Let's get started!");
    email3.send("user3@example.com");
```

```
    return 0;
}

import java.util.*;

interface EmailTemplate {
    void setContent(String content);
    void send(String to);
}

// A concrete email class, hardcoded
class WelcomeEmail implements EmailTemplate {
    private String subject;
    private String content;

    public WelcomeEmail() {
        this.subject = "Welcome to TUF+";
        this.content = "Hi there! Thanks for joining us.";
    }

    @Override
```

```
    public void setContent(String content) {
        this.content = content;
    }

    @Override
    public void send(String to) {
        System.out.println("Sending to " + to + ": [" + subject + "] " + content);
    }
}

class Main {
    public static void main(String[] args) {
        // Create a welcome email
        WelcomeEmail email1 = new WelcomeEmail();
        email1.send("user1@example.com");

        // Suppose we want a similar email with slightly different content
        WelcomeEmail email2 = new WelcomeEmail();
        email2.setContent("Hi there! Welcome to TUF Premium.");
        email2.send("user2@example.com");
```

```
        // Yet another variation
        WelcomeEmail email3 = new WelcomeEmail();
        email3.setContent("Thanks for signing up. Let's get started!");
        email3.send("user3@example.com");
    }
}
```

```
from abc import ABC, abstractmethod

# Target Interface
# Abstract class to represent an Email Template
class EmailTemplate(ABC):
    @abstractmethod
    def set_content(self, content):
        pass

    @abstractmethod
    def send(self, to):
        pass

# A concrete email class, hardcoded
```

```
class WelcomeEmail(EmailTemplate):
    def __init__(self):
        self.subject = "Welcome to TUF+"
        self.content = "Hi there! Thanks for joining us."

    def set_content(self, content):
        self.content = content

    def send(self, to):
        print(f"Sending to {to}: [{self.subject}] {self.content}")

if __name__ == "__main__":
    # Create a welcome email
    email1 = WelcomeEmail()
    email1.send("user1@example.com")

    # Suppose we want a similar email with slightly different content
    email2 = WelcomeEmail()
    email2.set_content("Hi there! Welcome to TUF Premium.")
    email2.send("user2@example.com")
```

```
# Yet another variation
email3 = WelcomeEmail()
email3.set_content("Thanks for signing up. Let's get started!")
email3.send("user3@example.com")
```

Issues in the Bad design

- **Tight Coupling to Concrete Class:**
  - The code uses the WelcomeEmail class directly.
  - No abstraction for cloning—client code is tightly bound to object creation logic (new WelcomeEmail() everywhere).
- **Repetitive Instantiation:**
  - For every variation, a new instance is created using the constructor—even though most data remains the same.
  - This leads to unnecessary duplication of code and logic.
- **Violates DRY Principle:** Repeated calls to new WelcomeEmail() and then setContent() for slight modifications break the Don't Repeat Yourself principle.
- **No Cloning or Copy Mechanism:** There is no concept of cloning or reusing a pre-defined template and just modifying small parts.

Good Code (Prototype Pattern Applied)

```
#include <bits/stdc++.h>
using namespace std;

// Defining the Prototype Interface
```

```
class EmailTemplate {
public:
    virtual EmailTemplate* clone() const = 0; // Recommended to perform deep copy
    virtual void setContent(const string& content) = 0;
    virtual void send(const string& to) const = 0;
    virtual ~EmailTemplate() {}
};

// Concrete Class implementing clone logic
class WelcomeEmail : public EmailTemplate {
private:
    string subject;
    string content;

public:
    WelcomeEmail() {
        subject = "Welcome to TUF+";
        content = "Hi there! Thanks for joining us.";
    }

    WelcomeEmail(const WelcomeEmail& other) {
```

```
        subject = other.subject;
        content = other.content;
    }

    EmailTemplate* clone() const override {
        return new WelcomeEmail(*this);
    }

    void setContent(const string& content) override {
        this->content = content;
    }

    void send(const string& to) const override {
        cout << "Sending to " << to << ": [" << subject << "] " << content << endl;
    }
};

// Template Registry to store and provide clones
class EmailTemplateRegistry {
private:
    static unordered_map<string, EmailTemplate*> templates;
```

```
public:
    static void init() {
        templates["welcome"] = new WelcomeEmail();
        // templates["discount"] = new DiscountEmail();
        // templates["feature-update"] = new FeatureUpdateEmail();
    }

    static EmailTemplate* getTemplate(const string& type) {
        return templates[type]->clone(); // clone to avoid modifying original
    }

    static void cleanup() {
        for (auto& pair : templates) {
            delete pair.second;
        }
    }
};

unordered_map<string, EmailTemplate*>
EmailTemplateRegistry::templates;
```

```
int main() {
    EmailTemplateRegistry::init();

    EmailTemplate* welcomeEmail1 =
EmailTemplateRegistry::getTemplate("welcome");
    welcomeEmail1->setContent("Hi Alice, welcome to TUF Premium!");
    welcomeEmail1->send("alice@example.com");

    EmailTemplate* welcomeEmail2 =
EmailTemplateRegistry::getTemplate("welcome");
    welcomeEmail2->setContent("Hi Bob, thanks for joining!");
    welcomeEmail2->send("bob@example.com");

    // Reuse the base WelcomeEmail structure, just changing dynamic
content

    delete welcomeEmail1;
    delete welcomeEmail2;
    EmailTemplateRegistry::cleanup();

    return 0;
}
```

```
import java.util.*;

// Defining the Prototype Interface
interface EmailTemplate extends Cloneable {
    EmailTemplate clone(); // Recommended to perform deep copy
    void setContent(String content);
    void send(String to);
}

// Concrete Class implementing clone logic
class WelcomeEmail implements EmailTemplate {
    private String subject;
    private String content;

    public WelcomeEmail() {
        this.subject = "Welcome to TUF+";
        this.content = "Hi there! Thanks for joining us.";
    }

    @Override
```

```
    public WelcomeEmail clone() {
        try {
            return (WelcomeEmail) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException("Clone failed", e);
        }
    }

    @Override
    public void setContent(String content) {
        this.content = content;
    }

    @Override
    public void send(String to) {
        System.out.println("Sending to " + to + ": [" + subject + "] " + content);
    }
}

// Template Registry to store and provide clones
class EmailTemplateRegistry {
```

```
    private static final Map<String, EmailTemplate> templates = new
HashMap<>();

    static {
        templates.put("welcome", new WelcomeEmail());
        // templates.put("discount", new DiscountEmail());
        // templates.put("feature-update", new FeatureUpdateEmail());
    }

    public static EmailTemplate getTemplate(String type) {
        return templates.get(type).clone(); // clone to avoid modifying
original
    }
}

// Driver code
class Main {
    public static void main(String[] args) {
        EmailTemplate welcomeEmail1 =
EmailTemplateRegistry.getTemplate("welcome");
        welcomeEmail1.setContent("Hi Alice, welcome to TUF Premium!");
        welcomeEmail1.send("alice@example.com");
```

```
        EmailTemplate welcomeEmail2 =
EmailTemplateRegistry.getTemplate("welcome");
        welcomeEmail2.setContent("Hi Bob, thanks for joining!");
        welcomeEmail2.send("bob@example.com");

        // Reuse the base WelcomeEmail structure, just changing dynamic
content
    }
}

import copy

# Defining the Prototype Interface
class EmailTemplate:
    def clone(self):
        return copy.deepcopy(self)  # Recommended to perform deep copy

    def set_content(self, content):
        pass

    def send(self, to):
```

```
        pass


# Concrete Class implementing clone logic
class WelcomeEmail(EmailTemplate):
    def __init__(self):
        self.subject = "Welcome to TUF+"
        self.content = "Hi there! Thanks for joining us."

    def set_content(self, content):
        self.content = content

    def send(self, to):
        print(f"Sending to {to}: [{self.subject}] {self.content}")

# Template Registry to store and provide clones
class EmailTemplateRegistry:
    templates = {
        "welcome": WelcomeEmail(),
        # "discount": DiscountEmail(),
        # "feature-update": FeatureUpdateEmail(),
    }
```

```
    @staticmethod
    def get_template(type_):
        return EmailTemplateRegistry.templates[type_].clone()  # clone to avoid modifying original

# Driver code
if __name__ == "__main__":
    welcome_email1 = EmailTemplateRegistry.get_template("welcome")
    welcome_email1.set_content("Hi Alice, welcome to TUF Premium!")
    welcome_email1.send("alice@example.com")

    welcome_email2 = EmailTemplateRegistry.get_template("welcome")
    welcome_email2.set_content("Hi Bob, thanks for joining!")
    welcome_email2.send("bob@example.com")

    # Reuse the base WelcomeEmail structure, just changing dynamic content
```

Benefits of Good Design

- **Implements clone():** Allows object copying instead of recreation.

- **Introduces Registry:** Central location (`EmailTemplateRegistry`) holds template prototypes.
- **Decouples creation from usage:** Client code doesn't depend on how `WelcomeEmail` is constructed.
- **Improves performance:** Avoids complex re-initialization logic by cloning pre-configured templates.

Deep Cloning VS Shallow Cloning

There are two types of cloning in Java: **Shallow Cloning** and **Deep Cloning**.

In the context of the Prototype Pattern, **Deep Cloning** is often preferred. This means that when you clone an object, not only the object itself is copied, but also all the objects it references. This ensures that changes to the cloned object do not affect the original object or any of its referenced objects.

Deep cloning is considered safer as well than shallow cloning because it avoids unintended side effects and ensures each clone is truly independent — especially important when templates contain complex internal structures (like nested configuration objects, lists, etc.).

Pros of Prototype Pattern

- **Faster object creation:** No need to reinitialize objects from scratch.
- **Reduces subclassing:** No need to create multiple subclasses for variations.
- **Runtime object configuration:** Easy to modify a clone on the fly.
- **Ideal for UI/UX cloning:** Useful when duplicating component trees or screen states.

Cons of Prototype Pattern

- **Deep cloning can be hard:** Implementing a true deep copy takes extra effort.
- **Trouble with circular references:** Cloning objects that refer to each other can lead to complex issues.
- **Potential for bugs:** If cloning isn't handled carefully, it may introduce unexpected behavior.

Class Diagram

The class diagram below illustrates the structure of the Prototype Pattern, showing how the various components interact with each other. Only the specification perspective is shown here, as the implementation perspective is not relevant for this pattern.